

Project2_Monopoly

Philopatair Mallik

CSC 17A _ C++

Spring 2026

42419

Introduction:

For project 2, I modified Project 1 that was a Monopoly Game. I added classes, Inheritance, Polymorphism, Class Templates.

About the game:

G

Monopoly is one if not he most popular board game around the world. It was made as a mockery of the capitalist system but families and people of all ages found out that is enjoyable to play and to rage at all the frustration that comes with the fun.

For Rules and a detailed account for the game here is a link to the game wiki:

https://monopoly.fandom.com/wiki/Main_Page

Github Link:

https://github.com/philomallik/CSC_CIS_17A_SPRING2026/tree/512187433c4be827384da7a5693e9ab1534ded1f/Projects/Project_2

Versions:

Version 0:

- Switched all structures from Previous Project to classes with appropriate member variables and functions, including constructors, destructors, accessors, mutators, operator overloads, and other member functions as needed for the Monopoly game implementation.
- Added header files for each class and implemented the member functions in corresponding .cpp files.
 - i have class Board with an array of pointers to square objects, and a function to initialize the board with the appropriate spaces and properties.
 - i have class Player with member variables for name, ID, token, balance, position, properties owned, jail status, and functions to move the player, buy properties, go to jail, etc.
 - i have class Dice with member variables for the two dice values and a counter for consecutive doubles, and functions to roll the dice, check for doubles, etc.
 - i have class Square as a base class for different types of spaces on the board, with derived classes for properties, action spaces, etc.
 - i have a class Property that inherits from Square and has additional member variables for rent, owner, etc., and functions to handle buying, paying rent, etc.

- i have a class City that inherits from Property and has additional member variables for the number of houses/hotels, and functions to handle upgrading the property, calculating rent, etc.
- i have a class Railroad that inherits from Property and has additional member variables for rent based on the number of railroads owned, and functions to calculate rent accordingly.
- i have a class Utility that inherits from Property and has additional member variables for rent multiplier based on the number of utilities owned, and functions to calculate rent based on dice rolls.
- i have a class ActionSpace that inherits from Square and has member variables for the type of action (e.g., Chance, Community Chest, Go to Jail, etc.) and functions to handle the action when a player lands on it.
- Modified the main game loop in Monopoly_V1.cpp to incorporate the new class structures and member functions, allowing players to take their turns, roll dice, move around the board, buy properties, pay rent, go to jail, etc., based on the new class implementations.

Version 1:

- Added the class file for the game board (Board) which contains an array of square objects representing the spaces on the board, and functions to initialize the board and read board data from an external file.
- Added the class file for the dice (Dice) which simulates rolling two six-sided dice and includes functionality to track consecutive doubles rolled by a player.
- Added the class file for the players (Player) which includes attributes such as name, ID, token, balance, position on the board, properties owned, jail status, and functions to handle player actions such as moving, buying properties, going to jail, etc.
- Added the class file for the properties (Property) which includes attributes such as name, price, rent, owner, and functions to handle property-related actions such as buying, paying rent, etc.
- Added the class file for the city properties (City) which inherits from Property and includes additional attributes such as color group, house count, house cost, and functions to handle building houses and calculating rent based on the number of houses.
- Added the class file for the railroads (Railroad) which inherits from Property and includes additional attributes such as rent based on the number of railroads owned, and functions to calculate rent accordingly.
- Added the class file for the utilities (Utility) which inherits from Property and includes additional attributes such as rent multiplier based on the number of utilities owned, and functions to calculate rent based on the dice roll.

- Updated the main game loop to incorporate the new classes and their functionalities, including player turns, dice rolls, and interactions with the board spaces.
- Board class is an array of square objects, where each Square can be a Property, Action Space, or other types of spaces on the Monopoly board.
- The Board class includes functions to initialize the board with the appropriate spaces and properties, and to read board data from an external file to set up the game.

Version 2:

- Added the function prototype for reading the board configuration from a file and initializing the board accordingly (readFromFile).
- Added the function prototype for moving a player a specified number of steps on the board (movePlayer), which will be used to handle player movement based on dice rolls and other game actions.
- Updated the main function to call the readFromFile function to initialize the board at the start of the game, and to include a placeholder for the movePlayer function to be implemented later.
- Added the necessary include statements for the new classes (Railroad and Utility) that were added in Version 1, to ensure that the main game file has access to these classes for handling player interactions with these types of properties on the board.
- Added comments to the main function and the new function prototypes to clarify their purpose and how they fit into the overall game implementation.

Version 3:

- Added Operator overloads for the Player class to allow for easier manipulation of the player's balance using += and -= operators, which will be used to handle transactions such as paying rent, receiving money, etc. in a more intuitive way in the game logic.
- Added a toString() member function to the Player class to print the details of the player (name, balance, position, properties owned, etc.) in a formatted manner, which will be useful for displaying player information during the game.

Version 4:

- Added the class file for the Chance and Community Chest cards (Card) which includes attributes such as description, action type, and value, and functions to handle the effects of the cards when drawn by a player.
 - Class Template for Deck<Card> storage of Chance and Community Chest cards has been implemented, allowing for easy management of the card decks in the game.

- class Card includes a constructor to initialize the card attributes, accessors to retrieve the card information, and a destructor.
- Started on implementing on using the card effects in the game, with placeholders for the logic to handle different types of card actions (e.g., moving to a specific square, paying or receiving money, getting out of jail free, etc.) when a player draws a card during their turn.
- Updated the main game loop to include logic for drawing cards when a player lands on a Chance or Community Chest space, and to call the appropriate functions to handle the card effects based on the type of card drawn.
- Added comments to the new class and function prototypes to clarify their purpose and how they fit into the overall game implementation.

Version 5:

- Added functionality to read and write the board configuration to a file, allowing the game state to be saved and loaded.
- Added manual initialization of the Chance and Community Chest decks to ensure the correct cards are available during gameplay.
- Fixed how the board is initialized in the main function to use the readFromFile function, and added a call to writeToFile to save the board configuration after initialization.
- Finished implementing the Chance and Community Chest card effects in the game logic, allowing players to experience the full range of card actions during their turns.
-